Informe técnico

Reproductor de video 360° para Meta Quest 3

Materia:	Realidad Virtual
Proyecto:	Vr360Video (Unity 6, URP y Meta XR SDK)
Repositorio:	github.com/AlberthGD/Vr360Video
Fecha:	Octubre de 2026

1. Descripción general

La aplicación reproduce videos esféricos en el Meta Quest 3. Al abrirla, el usuario ve el menú flotando sobre su habitación (passthrough) y elige uno de los recorridos. El video se proyecta en el skybox de la cámara y durante la reproducción se puede abrir una consola con barra de progreso, pausa y regreso al menú.

En esta entrega se agregaron tres funciones: el catálogo de videos ahora se descarga de un servidor en formato JSON, se pueden reproducir videos estéreo Over Under y dentro del video aparecen puntos de interés (hotspots) que muestran información al seleccionarlos.

2. Arquitectura y diagrama de estados

Toda la lógica de la app pasa por `AppController`, que funciona como una máquina de estados con cuatro estados: `Menu`, `Loading`, `Playing` y `Paused`. Cada cambio de estado se hace en el método `CambiarEstado()`, donde se decide qué UI se muestra, cómo se configura la cámara y si se crean o eliminan los hotspots. Las transiciones entre el menú y el video tienen un fundido a negro con un `CanvasGroup` para que el cambio de la habitación real al video no sea brusco.

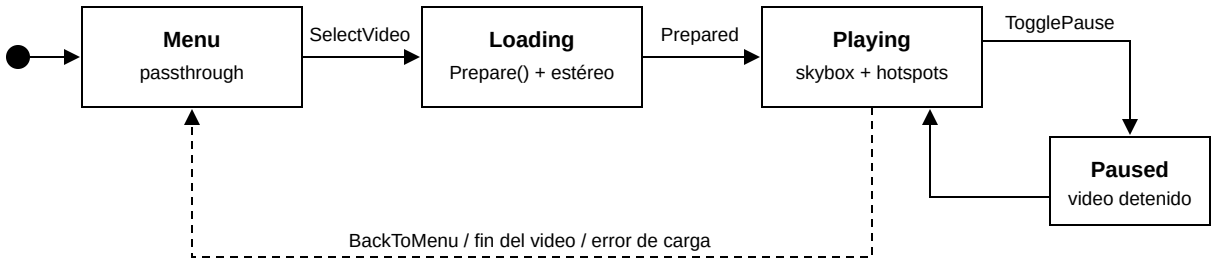


Figura 1. Estados de AppController.

En `Menu` y `Loading` la cámara usa `CameraClearFlags.SolidColor` con color transparente y `RenderSettings.skybox = null`, así se ve el passthrough. En `Playing` y `Paused` cambia a `CameraClearFlags.Skybox` con el material `360VideoMatt` (shader Skybox/Panoramic), que lee la `RenderTexture` donde el `VideoPlayer` dibuja el video. Si el video no se puede cargar (por ejemplo,

una URL remota sin conexión), `VideoService` recibe el error y la app vuelve al menú en lugar de quedarse en la pantalla de carga.

3. Clases del proyecto

Los scripts están en `Assets/Scripts/`. Con las funciones nuevas pasaron de 8 a 10 clases de ejecución, más un script de Editor.

Clase	Responsabilidad
`AppController`	Máquina de estados, fundidos, cambio de cámara y configuración del modo estéreo del skybox.
`VideoService`	Maneja el `VideoPlayer`: carga desde StreamingAssets o desde una URL, reproduce, pausa, adelanta y avisa cuando el video termina o falla.
`VideoCatalog`, `VideoEntry`, `HotspotData`, `CatalogData`	Datos del catálogo. El ScriptableObject se mantiene como respaldo y `CatalogData` es la estructura del JSON.
`RemoteCatalogLoader` (nueva)	Descarga `catalog.json` y las miniaturas, guarda una copia en caché y le pasa la lista de videos al menú.
`MenuView`	Crea un botón por cada video del catálogo y vuelve a pedir el catálogo al regresar al menú.
`VideoButton`	Muestra miniatura, título y duración; se agranda un poco cuando el rayo lo apunta.
`PlaybackControls`	Consola de reproducción que se abre con A, B, X o Y. Tiene la barra de progreso y los botones Pausar y Volver.
`HotspotManager` (nueva)	Crea los puntos de interés del video actual, detecta cuál apunta el mando y muestra el panel con la información.
`UIFollower`, `FPSCounter`	Seguimiento suave del canvas y contador de FPS en pantalla.
`Vr360Setup` (Editor)	Opción de menú *Vr360 > Configurar y validar escena*. Agrega y conecta los componentes nuevos y genera un reporte de validación.

4. Diseño de la interfaz

- **Menú sobre la habitación.** Preferimos que el usuario empiece viendo su espacio real con el passthrough y no dentro de un entorno virtual vacío. Así sabe dónde está antes de entrar al video.
- **Catálogo.** Es un canvas en World Space (escala 0.002) a unos 2 m del usuario, con una tarjeta por video. Las tarjetas se agrandan a 1.1x al apuntarlas con el rayo para que se note cuál está

seleccionada.

- **Consola de reproducción.** Mientras se ve el video no hay nada en pantalla. Al presionar A, B, X o Y aparece frente a la mirada un panel de 800×150 px con la barra de progreso y los botones circulares de Pausar/Reanudar y Volver.
- **Seguimiento del canvas.** El canvas sigue la cabeza con `Vector3.Lerp` y `Quaternion.Slerp`, y solo gira en el plano horizontal para que no se incline ni tiemble.
- **Hotspots.** Son esferas pequeñas de color celeste que pulsan suavemente, con su nombre debajo. Al apuntarlas se ponen amarillas y se agrandan; con el gatillo se abre un panel oscuro con el título y la descripción, un poco por debajo del punto para no taparlo. El panel se cierra con el gatillo apuntando a otra parte o solo a los 10 segundos.

5. Funciones agregadas

5.1 Catálogo en JSON descargado desde un servidor

Antes los videos estaban definidos en el ScriptableObject `Catalogo`, así que para agregar uno había que modificarlo en Unity y volver a compilar el APK. Ahora `RemoteCatalogLoader` descarga el archivo `catalog.json` con `UnityWebRequest` al iniciar la app y cada vez que se regresa al menú. Para agregar un video basta con editar el JSON en el servidor y volver al menú en el visor.

Si la descarga falla, la app no se queda sin catálogo. Se intenta en este orden:

1. Servidor (por defecto `raw.githubusercontent.com/AlberthGD/Vr360Video/main/Server/catalog.json`).
2. Copia de la última descarga guardada en `Application.persistentDataPath`.
3. `StreamingAssets/catalog.json`, incluido en el APK.
4. El ScriptableObject original.

Cada video puede usar `fileName` (archivo dentro de StreamingAssets) o `url` (video en línea que se reproduce por streaming). Las miniaturas se descargan desde `thumbnailUrl`; si no se pueden descargar, se usa el sprite local del video con el mismo nombre. A la URL del catálogo se le agrega un parámetro con la hora para evitar que se lea una versión vieja guardada en caché.

```
{
  "title": "CampusU",
  "fileName": "CampusU.mp4",
  "thumbnailUrl": "https://raw.githubusercontent.com/.../thumbs/CampusUIMG.png",
  "duration": "1:13",
  "loop": false,
  "stereo": "Mono",
  "hotspots": [
    { "title": "Edificio principal", "description": "...",
      "yaw": 0, "pitch": 5, "startTime": 0, "endTime": -1 }
  ]
}
```

Para el servidor hay dos opciones: subir la carpeta `Server/` al repositorio y leerla desde GitHub por HTTPS, o levantar un servidor local en la PC con `python -m http.server 8000` en la misma red que el visor. En el segundo caso hay que activar *Allow downloads over HTTP* en Player Settings, porque Android bloquea HTTP sin cifrar.

5.2 Soporte de videos estéreo Over Under

En un video Over Under cada cuadro trae dos imágenes: la de arriba para el ojo izquierdo y la de abajo para el derecho. El shader Skybox/Panoramic de Unity ya trae esa opción en su propiedad `_Layout` (0 = mono, 1 = side by side, 2 = over under) y usa `unity_StereoEyeIndex` para darle a cada ojo su mitad.

En el JSON cada video tiene el campo `"stereo"` con `"Mono"`, `"OverUnder"` o `"SideBySide"`. Antes de cargar el video, `AppController.ConfigurarEstereo()` convierte ese texto al valor de `_Layout` y lo asigna al material. Al iniciar la app se hace una copia del material con `new Material()`, para no modificar el archivo `360VideoMatt.mat` del proyecto cada vez que se cambia de modo.

Los tres videos que tenemos son monoscópicos, por eso para probar esta función hay que agregar al catálogo un video grabado en Over Under. Si el video es de alta resolución conviene subir la `RenderTexture` de 4096×2048 a 4096×4096, porque en este formato cada ojo usa solo la mitad de la altura.

5.3 Puntos de interés (hotspots)

Los hotspots se definen en el mismo JSON, dentro de cada video. Cada uno tiene título, descripción, posición en grados (`yaw` horizontal y `pitch` vertical, medidos desde el centro del video) y el intervalo de tiempo en que se muestra (`startTime` y `endTime`, donde -1 significa hasta el final).

Cuando el video pasa a `Playing`, `HotspotManager` crea una esfera con collider por cada punto. Como el skybox está "en el infinito", las esferas se colocan a 6 m de la cámara en la dirección indicada y se mueven con la cabeza en cada frame. Para pasar de yaw/pitch a una dirección hubo que revisar el shader: Skybox/Panoramic pone el centro de la imagen en el eje +X y no en +Z, por eso la fórmula usada es:

```
direccion = Quaternion.Euler(-pitch, 90 + rotacionSkybox + yaw, 0) * Vector3.forward
```

La selección se hace con `Physics.RaycastAll` desde el `RightControllerAnchor` del `OVRCameraRig` (o el izquierdo si solo está ese mando) y el gatillo abre el panel. En el Editor también funciona con el mouse. Para no tener que adivinar las coordenadas, agregamos una herramienta: al presionar el joystick derecho (o la tecla C en el Editor) aparece el yaw, pitch y tiempo hacia donde apunta el rayo, listos para copiar al JSON.

6. Rendimiento

Para medir los FPS dentro del visor, sin depender del cable Link, se hizo `FPSCounter.cs`. Cada 0.5 s calcula los cuadros por segundo con `Time.unscaledDeltaTime` y los muestra en verde (70 o más), amarillo (50 a 69) o rojo (menos de 50).

Situación	FPS	Tiempo por cuadro
Menú sobre passthrough	72	13.88 ms
Reproducción 360° con la consola abierta	72	13.88 ms

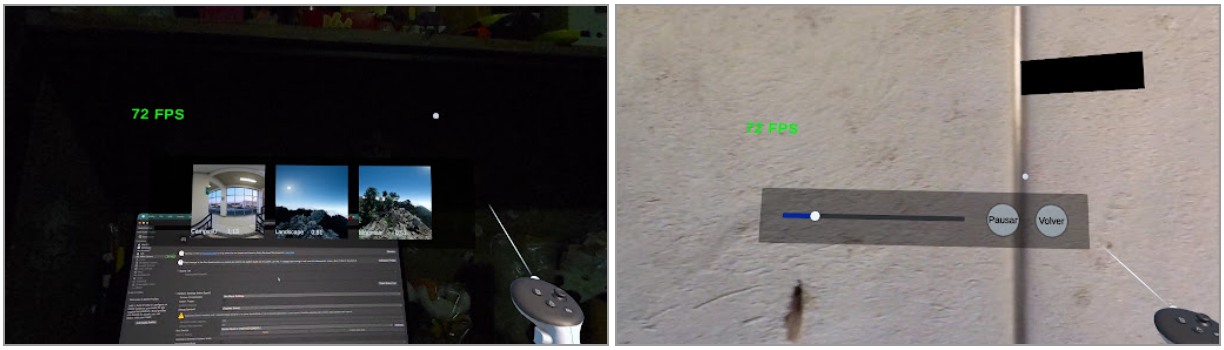


Figura 2. Izquierda: menú con los tres videos sobre la habitación. Derecha: consola de reproducción durante el video. En ambos casos el contador marca 72 FPS.

Lo que más ayudó a mantener los 72 Hz fue leer los videos por streaming desde `Application.streamingAssetsPath` en lugar de cargarlos completos en memoria, usar una sola `RenderTexture` que se limpia con `GL.Clear` entre videos, y trabajar en URP sin sombras dinámicas ni geometría extra. Los hotspots agregan pocos objetos (esferas simples sin sombras y un material Sprites/Default), así que no deberían afectar el rendimiento. Aun así, las mediciones de la tabla se tomaron antes de agregar las funciones nuevas y hay que repetirlas en el visor.

7. Problemas encontrados y soluciones

Problema	Causa	Solución

Los botones dejaban de responder al láser después de mover la consola.	Solo se movía el subpanel `PanelControlesVideo` y sus botones quedaban fuera del área del `PointableCanvas` padre.	En `PlaybackControls` se mueve y rota la raíz del canvas completo.
El video 360° no se veía con el passthrough activado.	Con la cámara en `SolidColor` transparente Unity no dibujaba el skybox.	En `CambiarEstado()` la cámara cambia entre `SolidColor` (menú) y `Skybox` (video).
La barra de progreso regresaba sola al adelantar.	El seek del `VideoPlayer` es asíncrono y `Update()` leía el tiempo anterior por unos milisegundos.	`EventTrigger` con PointerDown/PointerUp y una espera de 0.8 s antes de volver a actualizar la barra, usando `SetValueWithoutNotify`.
La consola no se abría con A/B.	Se desactivaba el mismo GameObject que tenía el script, y con eso se detenía su `Update()`.	El script quedó en `AppManager`, que siempre está activo, y la parte visual en la referencia `panelVisual`.
No se podía subir el proyecto a GitHub.	Los tres videos suman cerca de 1 GB y GitHub no acepta archivos de más de 100 MB.	Git LFS para `*.mp4` y un `.gitignore` que excluye `Library/` y `Temp/`.
Los hotspots aparecían en un lugar distinto al esperado.	Supusimos que el centro del video estaba en +Z, pero el shader Skybox/Panoramic lo pone en +X.	Se sumaron 90° al yaw (más la rotación del material) y se comprobó con una prueba de ida y vuelta yaw/pitch → dirección → yaw/pitch.
Al pulsar Pausar o Volver también se abría un hotspot.	El gatillo hace clic en la UI y al mismo tiempo el rayo de los hotspots podía estar tocando una esfera detrás de la consola.	Mientras la consola está abierta, `HotspotManager` ignora el gatillo.
Los videos no se podían reproducir desde la URL de GitHub.	raw.githubusercontent devuelve el archivo puntero de Git LFS, no el .mp4.	Desde GitHub solo se leen el JSON y las miniaturas. Los videos remotos se sirven desde la PC u otro hosting.
El script de configuración agregaba componentes repetidos.	En el Editor, `GetComponent` devuelve un "null falso" y el operador `??` no lo detecta.	Se cambió por una comparación normal con `== null`.

8. Validación

La opción *Vr360 > Configurar y validar escena* revisa la escena y deja el resultado en `Logs/Vr360Validacion.txt`. En la última ejecución todo salió correcto:

- `RemoteCatalogLoader` y `HotspotManager` están en `AppManager` con todas sus referencias.

- Los dos `catalog.json` (StreamingAssets y Server) se leen bien: 3 videos con su archivo presente y 2 hotspots cada uno.
- El material del skybox tiene la propiedad `_Layout` y los valores `"Mono"`, `"SideBySide"` y `"OverUnder"` se convierten a 0, 1 y 2.
- La conversión de coordenadas de los hotspots no tiene error y el centro del video quedó en (1, 0, 0).

Lo que falta comprobar en el visor: que el rayo de los hotspots coincida con el láser del mando (si no, se corrige con `correccionPitchRayo`), las coordenadas reales de cada punto en los videos y un video Over Under para la prueba de estéreo.

9. Fuentes y licencias del contenido

Archivo	**Duración**	**Origen**	**Licencia**
`CampusU.mp4`	01:13	Grabación 360° propia, hecha en el campus universitario.	Material propio, uso libre en el proyecto.
`Landscape.mp4`	00:51	Video de muestra incluido con Insta360 Studio.	Material de demostración de Insta360.
`Mountain.mp4`	00:51	Video de muestra incluido con Insta360 Studio.	Material de demostración de Insta360.